Q.1.In an e-commerce system, customer account IDs are stored in a list, and you are tasked with writing a program that implements the following: • Linear Search: Check if a particular customer account ID exists in the list. • Binary Search: Implement Binary search to find if a customer account ID exists, improving the search efficiency over the basic linear.

```
ids = [10, 20, 30, 40, 50]


# Linear Search
def linear_search(key):
    for i in ids:
        if i == key:
            return "Found"
    return "Not Found"


# Binary Search
def binary_search(key):
    low = 0
    high = len(ids) - 1

    while low <= high:
        mid = (low + high) // 2

        if ids[mid] == key:
            return "Found"
        elif ids[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return "Not Found"


key = int(input("Enter customer ID: "))
```

```
print("Linear Search:", linear_search(key))

print("Binary Search:", binary_search(key))
```

Q.2. Problem statement: In a company, employee salaries are stored in a list as floating-point numbers. Write a python program that sorts the employee salaries in ascending order using following two algorithms: 1. Selection sort: Sort the values using the selection sort algorithm 2. Bubble sort: Sort the salaries using bubble sort algorithm After sorting the salaries, the program should display top 5 highest salaries in the company.

```
salary = [25000.5, 40000.0, 15000.0, 60000.0, 35000.0, 50000.0]


# Selection Sort

a = salary.copy()


for i in range(len(a)):
    min = i
    for j in range(i+1, len(a)):
        if a[j] < a[min]:
            min = j
    a[i], a[min] = a[min], a[i]


print("Selection Sort:", a)

print("Top 5 Salaries:", a[-5:][::-1])


# Bubble Sort

b = salary.copy()


for i in range(len(b)):
    for j in range(len(b)-1-i):
        if b[j] > b[j+1]:
            b[j], b[j+1] = b[j+1], b[j]
```

```
print("Bubble Sort:", b)
print("Top 5 Salaries:", b[-5:][::-1])
```

Q.3. Implementing a real-time undo/redo system for a text editing application using a Stack data structure. The system should support the following operations: - Make a Change: A new change to the document is made. - Undo Action: Revert the most recent change and store it for potential redo. - Redo Action: Reapply the most recently undone action. - Display Document State: Show the current state of the document after undoing or redoing an action

```
undo = []
redo = []
text = ""

# Make a change
def change(new_text):
    global text
    undo.append(text)
    text = new_text
    redo.clear()

# Undo
def undo_action():
    global text
    if undo:
        redo.append(text)
        text = undo.pop()
    else:
        print("Nothing to undo")

# Redo
def redo_action():
```

```
    global text
    if redo:
        undo.append(text)
        text = redo.pop()
    else:
        print("Nothing to redo")

# Display
def display():
    print("Document:", text)



# Main
change("Hello")
change("Hello World")
display()

undo_action()
display()

redo_action()
display()
```

Q.4.Implement a real-time event processing system using a Queue data structure. The system should support the following features: - Add an Event: When a new event occurs, it should be added to the event queue. - Process the Next Event: The system should process and remove the event that has been in the queue the longest. - Display Pending Events: Show all the events currently waiting to be processed. - Cancel an Event: An event can be canceled if it has not been processed

```
queue = []
```

```
# Add Event
def add_event(event):
    queue.append(event)
    print("Event added")

# Process Event
def process_event():
    if queue:
        print("Processed:", queue.pop(0))
    else:
        print("Queue is empty")

# Display Events
def display():
    print("Pending Events:", queue)

# Cancel Event
def cancel_event(event):
    if event in queue:
        queue.remove(event)
        print("Event cancelled")
    else:
        print("Event not found")


# Main
add_event("Event 1")
add_event("Event 2")
add_event("Event 3")
```

```
display()


process_event()


cancel_event("Event 3")


display()
```

Q.5. Write a Python program to manage the borrowing records of books in a library. Implement the following functionalities: • Compute the average number of books borrowed by all library members. • Find the book with the highest and lowest number of borrowings in the library. • Count the number of members who have not borrowed any books (denoted by a borrow count of 0). • Display the most frequently borrowed book (i.e., the mode of borrow counts). After performing, determine the time and Space complexity of each operation

```
# Library Borrowing Records


n = int(input("Enter number of members: "))


books = []


# Accept borrowing records

for i in range(n):

    x = int(input("Enter books borrowed: "))

    books.append(x)


# 1. Average

total = sum(books)

print("Average:", total / n)


# 2. Highest and Lowest

print("Highest borrowing:", max(books))

print("Lowest borrowing:", min(books))
```

```
# 3. Count members who borrowed zero books
zero = 0

for x in books:
    if x == 0:
        zero = zero + 1

print("Members with zero borrowing:", zero)

# 4. Find Mode
mode = books[0]
max_count = 0

for x in books:
    count = books.count(x)

    if count > max_count:
        max_count = count
        mode = x

print("Most frequently borrowed:", mode)
```

Q.6. Implement a hash table of size 10 and use the division method as a hash function. In case of a collision, use chaining. Implement the following operations: • Insert(key): Insert key-value pairs into the hash table. • Search(key): Search for the value associated with a given key. • Delete(key): Delete a key-value pair from the hash table

```
table = [[] for i in range(10)]

# Insert
def insert(key, value):
```

```
    index = key % 10
    table[index].append([key, value])
    print("Inserted")

# Search
def search(key):
    index = key % 10
    for pair in table[index]:
        if pair[0] == key:
            return pair[1]
    return "Not Found"

# Delete
def delete(key):
    index = key % 10
    for pair in table[index]:
        if pair[0] == key:
            table[index].remove(pair)
            print("Deleted")
            return
    print("Key not found")

# Display
def display():
    for i in range(10):
        print(i, ":", table[i])



# Main
insert(10, "Apple")
insert(20, "Banana")
```

```
insert(15, "Mango")


display()


print("Search:", search(20))


delete(10)


display()
```

Q.7. Consider a particular area in your city. Note the popular locations A, B, C... in that area. Assume these locations represent nodes of a graph. If there is a route between two locations, it is represented as connections between nodes. Find out the sequence in which you will visit these locations, starting from (say A) using (i) BFS and (ii) DFS. Represent a given graph using an adjacency matrix to perform DFS and an adjacency list to perform BFS.

```
# Graph using Adjacency List for BFS
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}


# BFS
def bfs(start):
    visited = []
    queue = [start]
```

```
    while queue:
        node = queue.pop(0)

        if node not in visited:
            visited.append(node)
            for i in graph[node]:
                if i not in visited:
                    queue.append(i)

    print("BFS:", visited)



# Graph using Adjacency Matrix for DFS
matrix = [
    [0, 1, 1, 0, 0, 0],
    [1, 0, 0, 1, 1, 0],
    [1, 0, 0, 0, 0, 1],
    [0, 1, 0, 0, 0, 0],
    [0, 1, 0, 0, 0, 1],
    [0, 0, 1, 0, 1, 0]
]

nodes = ['A', 'B', 'C', 'D', 'E', 'F']
visited = []

# DFS
def dfs(v):
    visited.append(nodes[v])
    for i in range(6):
        if matrix[v][i] == 1 and nodes[i] not in visited:
            dfs(i)
```

```
bfs('A')

dfs(0)

print("DFS:", visited)
```

Q.8.Implement various operations on a Binary Search Tree, such as insertion, deletion, display, and search

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Insert
def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)
    else:
        root.right = insert(root.right, data)

    return root

# Search
def search(root, key):
    if root is None or root.data == key:
        return root
```

```
    if key < root.data:
        return search(root.left, key)

    return search(root.right, key)

# Display (Inorder)
def display(root):
    if root:
        display(root.left)
        print(root.data, end=" ")
        display(root.right)

# Delete
def delete(root, key):
    if root is None:
        return root

    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left

        temp = root.right
        while temp.left:
            temp = temp.left
```

```
        root.data = temp.data
        root.right = delete(root.right, temp.data)

    return root


# Main
root = None

for x in [50, 30, 70, 20, 40, 60, 80]:
    root = insert(root, x)

print("BST:")
display(root)

print("\nSearch 40:", "Found" if search(root, 40) else "Not Found")

root = delete(root, 30)

print("After deletion:")
display(root)
```

Q.9. Create a Student Record Management System using LinkedList • Use a single/doubly linked list to store student data (Roll No, Name, Marks). • Perform operations: Add, Delete, Update, Search, and Sort. • Display records in ascending/descending order based on marks or roll number

```
class Node:
    def __init__(self, roll, name, marks):
        self.roll = roll
        self.name = name
```

```
        self.marks = marks
        self.next = None

head = None

# Add Student
def add(roll, name, marks):
    global head
    new = Node(roll, name, marks)
    new.next = head
    head = new

# Display
def display():
    temp = head
    while temp:
        print(temp.roll, temp.name, temp.marks)
        temp = temp.next

# Search
def search(roll):
    temp = head
    while temp:
        if temp.roll == roll:
            print("Found:", temp.name, temp.marks)
            return
        temp = temp.next
    print("Not Found")

# Delete
def delete(roll):
```

```
    global head
    temp = head
    prev = None

    while temp:
        if temp.roll == roll:
            if prev is None:
                head = temp.next
            else:
                prev.next = temp.next
            print("Deleted")
            return
        prev = temp
        temp = temp.next

    print("Not Found")

# Update
def update(roll, marks):
    temp = head
    while temp:
        if temp.roll == roll:
            temp.marks = marks
            print("Updated")
            return
        temp = temp.next
    print("Not Found")

# Sort by Marks
def sort():
    global head
```

```
    a = []
    temp = head

    while temp:
        a.append(temp)
        temp = temp.next

    a.sort(key=lambda x: x.marks)

    head = None
    for node in reversed(a):
        node.next = head
        head = node


# Main
add(1, "Amit", 85)
add(2, "Riya", 90)
add(3, "Neha", 75)

print("Student Records:")
display()

search(2)

update(1, 95)

delete(3)

sort()
```

```
print("Sorted Records:")
display()
```